

PROCESADORES DE LENGUAJE - PRÁCTICA FINAL

Grupo 206

09/05/2025

Jorge Mejías Donoso 100495807@alumnos.uc3m.es

Alberto Menchen Montero 100495692@alumnos.uc3m.es

1. Introducción	2
2. Declaraciones	2
3 Memoria del Traductor Frontend (trad.y)	3
3.1 Variables globales	3
3.2 Funciones y main	3
3.3 Impresión de cadenas con puts	4
3.4 Impresión de expresiones: printf	5
3.5 Operadores, precedencia y asociatividad	5
3.6 Estructura de control while	7
3.7 Estructura de control if	8
3.8 Estructura de control for	10
3.9.1 Asignaciones en funciones y en main	11
3.9.2 Declaraciones en funciones y en main	13
3.10 Definición de funciones auxiliares	15
3.11 Llamadas a funciones	17
4. Memoria del Traductor Backend (back.y)	18
4.1 Traducción de variables globales.	18
4.2 Traducción de funciones (main)	19
4.3 Traducción de expresiones aritméticas, comparaciones y operadores lógicos	20
4.4 Traducción de asignaciones locales (setf)	22
4.5 Traducción de if y if con else	23
4.6 Traducción de bucles while	26
4.7 Traducción de print, princ	27
5. Validación y pruebas	28
6. Conclusión final	29

1. Introducción

Este proyecto consiste en la implementación completa de un traductor de código fuente en lenguaje C hacia un lenguaje intermedio basado en LISP (frontend), seguido de un segundo traductor de ese código intermedio hacia código ejecutable en Forth (backend). El objetivo principal es comprender el funcionamiento interno de un compilador dividido en dos etapas, abordando tanto el análisis sintáctico como la generación de código, utilizando Bison como herramienta principal de construcción.

El trabajo se ha desarrollado en dos fases claramente diferenciadas:

- **Frontend** (`trad.y`): traduce código C a una representación intermedia en LISP. Se ha desarrollado con atención al diseño jerárquico de la gramática, uso controlado de recursividad y una semántica clara y modular.
- **Backend** (`back.y`): transforma el código LISP generado en una versión equivalente en lenguaje Forth, siguiendo el modelo de máquina de pila.

Durante el desarrollo se han utilizado herramientas como `flex`, `bison`, `clisp` y `gforth`, siguiendo las directrices metodológicas propuestas en las sesiones prácticas.

2. Declaraciones

Declaración de uso de herramientas de inteligencia artificial:

Durante el desarrollo del proyecto no se han utilizado herramientas de inteligencia artificial para la cumplimentación del código. En algunas ocasiones, intentamos hacerle alguna consulta en relación a algún error que nos aparecía, pero era completamente inútil su ayuda, por lo que nos ceñimos a hacerlo completamente a mano, desarrollando una gramática personalizada y clara para nuestro entendimiento.

Declaración de reparto equitativo:

Los dos autores han participado equitativamente en el desarrollo del proyecto. Se ha cooperado en todo momento para cada uno de los pasos del trabajo, ya que consideramos que era un proyecto con un grado de complejidad elevado y contaba con muchas partes dependientes, por lo tanto, si alguno de los dos no seguía el ritmo se iba a ver imposibilitado para entender la implementación.

3 Memoria del Traductor Frontend (**trad.y**)

Esta sección se estructura siguiendo los requisitos de especificación del enunciado (puntos 1 al 11), limitando la explicación a **1 página por apartado**, como recomienda el enunciado.

3.1 Variables globales

En el lenguaje C, las variables globales se declaran fuera de cualquier función, normalmente al inicio del programa. En nuestra implementación, estas se traducen directamente en sentencias Lisp de tipo (**setq id valor**), donde **valor** es **0** por defecto si no hay inicialización explícita. Esta conversión refleja la semántica de reserva de memoria e inicialización en C, adaptada a la sintaxis funcional prefija de Lisp.

Para ello, se han definido unas producciones específicas para la declaración de variables:

- Variables globales con las producciones **definicion_var_global** y **conjunto_vars_global**
- Variables estandarizadas con **declaracion_variable** y **lista_declaracion**.

Todo este conjunto de reglas comprendidas dentro de estas producciones completan la declaración de variables.

3.2 Funciones y **main**

La función **main** representa el punto de entrada de los programas escritos en C. Además, nuestro traductor también reconoce y traduce funciones auxiliares definidas por el usuario, con o sin parámetros. Todas las funciones se transforman a notación Lisp utilizando la forma (**defun nombre (params) cuerpo**), siguiendo un modelo estructurado y compatible con el backend.

Reconocimiento y estructura gramatical

Siguiendo la recomendación de mantener una gramática estructurada y jerárquica, se ha introducido una producción dedicada que:

- Detectar funciones con nombre **main** y sin parámetros.
- Reconocer funciones auxiliares con parámetros formales entre paréntesis.
- Traducir el cuerpo de las funciones utilizando **bloque_main** o **bloque_funcion**, que encapsulan las sentencias internas válidas: declaraciones locales, estructuras de control (**if**, **while**, **for**), llamadas a funciones, operaciones con variables, y funciones de impresión (**printf**, **puts**).

En el caso de `main`, se exige estrictamente la forma `main()` sin parámetros. Para las funciones auxiliares, se permite especificar una lista de parámetros, que se transforman en argumentos formales dentro del `defun`.

Restricciones y control semántico

- Se garantiza que solo puede haber una función `main` por programa.
- La ejecución del programa se realiza a través de la directiva especial `//@ (main)` insertada al final del código de entrada. Esta se traduce explícitamente como `(main)` y nunca se imprime automáticamente desde el axioma.
- Las funciones auxiliares pueden ser llamadas desde `main` o desde otras funciones, y deben definirse antes de ser invocadas si se desea evitar errores en la ejecución.
- No se admite `main` con parámetros como `main(int argc, char* argv[])`.

Validación

Se han verificado múltiples casos:

- Funciones `main` con varias sentencias secuenciales, estructuras condicionales, bucles y llamadas a otras funciones.
- Funciones auxiliares con uno o varios parámetros, llamadas desde `main`, y retornando valores o simplemente realizando acciones internas.
- Uso combinado de variables globales y locales en distintos contextos funcionales.

Todas las pruebas han confirmado el correcto comportamiento sintáctico y semántico de las funciones, generando código Lisp válido y compatible con la ejecución posterior en `clisp`.

3.3 Impresión de cadenas con `puts`

La función `puts` de C permite imprimir una cadena literal seguida de un salto de línea. En nuestro traductor, esta instrucción se convierte directamente en una llamada a `(print ...)` en Lisp, respetando el comportamiento funcional.

Reconocimiento sintáctico

Se ha implementado una producción específica que:

- Reconoce la palabra clave `puts`.
- Valida la presencia de una única cadena delimitada por comillas dobles.
- Requiere los paréntesis y el punto y coma finales.

La cadena se captura como un token `STRING`, procesado por `yyllex` mediante expresiones regulares específicas.

Transformación semántica

La cadena detectada se inserta como argumento en la función (`print ...`), lo que garantiza la impresión con salto de línea. Esta transformación se limita únicamente a cadenas literales sin variables ni expresiones.

Validación

Se han realizado pruebas unitarias con distintos textos, confirmando que el comportamiento en `clisp` reproduce correctamente la funcionalidad de `puts`. Además, se ha verificado que no se genera ninguna salida si el token `STRING` está mal formado, respetando la sintaxis.

3.4 Impresión de expresiones: `printf`

La instrucción `printf` de C, diseñada para mostrar expresiones formateadas, no imprimiendo la parte entre comillas como se especifica

Reconocimiento gramatical

La gramática permite:

- Una cadena de formato como primer argumento (descartada).
- Una lista de argumentos adicionales separados por comas, que pueden ser expresiones aritméticas o cadenas (`STRING`).
- El uso obligatorio de paréntesis y punto y coma.

Cada uno de estos argumentos se traduce a una sentencia individual (`princ ...`), que en Lisp permite imprimir el valor sin salto de línea.

Reglas semánticas aplicadas

- La cadena de formato se ignora por completo. No se interpreta ni se imprime.
- Cada argumento se traduce y se imprime de forma secuencial mediante `princ`.
- No se imprime salto de línea al final, respetando el comportamiento original de `printf` sin `\n`.

Validación

Se han probado expresiones combinadas, como variables, sumas, y cadenas, en distintos órdenes. Todas ellas se han evaluado correctamente en `clisp`, confirmando el correcto comportamiento de impresión secuencial.

3.5 Operadores, precedencia y asociatividad

Uno de los pilares del diseño gramatical de esta práctica es la correcta traducción de expresiones complejas con operadores aritméticos, lógicos y de comparación. En nuestro traductor, se ha implementado una gramática que respeta la **precedencia y asociatividad**

definidas por el lenguaje C, traduciéndolas correctamente a su versión en notación prefija LISP.

Organización gramatical basada en niveles

Siguiendo el diseño jerárquico recomendado en el PDF de la práctica, se ha estructurado la gramática por niveles de expresión, en un orden descendente de precedencia:

1. **Expresiones lógicas:** `&&`, `||`
2. **Comparaciones:** `==`, `!=`, `<`, `>`, `<=`, `>=`
3. **Aritmética:** `*`, `/`, `%`, `+`, `-`, (estas dos últimas con menor prioridad dentro de su nivel)
4. **Unarios:** `-` (negativo), `!` (negación lógica)

Esta separación permite un tratamiento claro y modular, evitando conflictos y manteniendo el control del flujo de traducción.

Traducción a notación prefija LISP

La traducción sigue una estructura de tipo:

Unset

```
a + b → (+ a b) | a && b → (and a b) | a == b → (= a b)
a % b → (mod a b) | -a → (- a) | !a → (not a)
```

Estas transformaciones se realizan mediante funciones semánticas que construyen las cadenas de salida prefijas al nivel de cada producción.

Resolución de precedencia con directivas de Bison

En lugar de forzar decisiones semánticas manuales, se ha aprovechado el sistema de **directivas de precedencia** de Bison (`%left`, `%right`) para evitar conflictos `shift/reduce`, simplificando el parser y manteniéndolo limpio y legible.

Compatibilidad con paréntesis

La gramática permite expresiones agrupadas mediante paréntesis (`(` y `)`), lo que fuerza la evaluación de subexpresiones de manera prioritaria y evita confusiones con la precedencia implícita.

Validación y pruebas

Se han realizado múltiples pruebas, muchas de ellas extraídas de `test.zip`, en las que se combinan operadores aritméticos, lógicos y de comparación. Todas las salidas coinciden con las que se obtendrían evaluando el código original en C.

3.6 Estructura de control **while**

La estructura de control **while** permite ejecutar repetidamente un bloque de código mientras se cumpla una determinada condición lógica. En el traductor, esta construcción se transforma en la instrucción **loop** de Lisp, con la forma:

```
(loop while <condición> do <bloque>).
```

Reconocimiento gramatical

La gramática implementada reconoce la estructura estándar de C:

```
C/C++  
while (<expresión>) {  
    <sentencias>  
}
```

Se exige la presencia explícita de los paréntesis y las llaves, tal como indica el enunciado. Dentro del bloque se permite cualquier conjunto de sentencias válidas: asignaciones, expresiones, impresiones, llamadas a funciones y estructuras anidadas.

Traducción semántica

La expresión de condición se traduce al formato prefijo de Lisp. El cuerpo del bucle se encierra dentro de un bloque **do**, que permite ejecutar todas las sentencias mientras se cumpla la condición.

Por ejemplo:

```
C/C++  
while (a != 10) {  
    a = a + 1;  
}
```

se traduce como:

```
Unset  
(loop while (/= a 10) do  
    (setf a (+ a 1))  
)
```

Se valida que la condición sea una expresión lógica correctamente formada, evitando construcciones no canónicas como **while (10 - a)** o valores constantes no booleanos, tal como especifica el enunciado.

Consideraciones técnicas

- Se prohíbe el uso de expresiones de tipo aritmético como condición si no derivan en una comparación válida. La gramática no acepta directamente condiciones numéricas que no puedan ser evaluadas como booleanas en Lisp.
- Se ha mantenido una estructura jerárquica donde `while` sólo puede aparecer dentro de funciones, nunca fuera de ellas.

Validación

Se han probado bucles simples, anidados y combinados con condicionales. También se han verificado casos límite como bucles con condición inicialmente falsa o variables no inicializadas. Todas las traducciones han generado código Lisp válido y funcional, comprobado con `clisp` y posteriormente con `gforth` en las pruebas del backend.

3.7 Estructura de control `if`

La estructura `if` permite ejecutar condicionalmente un bloque de código si se cumple una determinada expresión. En el traductor, esta instrucción se transforma directamente en la estructura condicional genérica de Lisp explicada más adelante

Reconocimiento gramatical

La gramática reconoce dos formas válidas de la estructura `if` en C:

```
C/C++
if (<condición>) {
    <bloque_if>
}

if (<condición>) {
    <bloque_if>
} else {
    <bloque_else>
}
```

Ambas versiones requieren expresamente llaves `{}` para delimitar los bloques, evitando ambigüedades y simplificando el análisis sintáctico.

Traducción semántica

En Lisp, la instrucción `if` acepta una condición y uno o dos bloques (este último haciendo referencia a la parte del `else`), escritos en notación prefija. En ambos dos bloques se ha contemplado el caso genérico por lo cual el programa siempre escribe `progn` al principio de cada bloque

Sin else:

```
Unset
(if (<condición>)
    (progn <sentencia_1>...<sentencia_n>)
)
```

Con else:

```
Unset
(if (<condición>)
    (progn <sentencia_1>...<sentencia_n>)
    (progn <sentencia_1>...<sentencia_n>)
)
```

Consideraciones técnicas

- La condición debe evaluarse como una expresión lógica válida (comparaciones, negaciones, operadores booleanos).
- No se admite `else if` directamente. Estas estructuras deben reescribirse como `if` anidados.
- Las expresiones condicionales se construyen previamente mediante las reglas de precedencia definidas en el apartado 3.5.

Manejo de expresiones booleanas

Dado que en Lisp no existe un tipo booleano estricto como en C, se ha implementado una conversión sistemática de expresiones como `a != 0` o `x < y` a sus equivalentes Lisp (`/=` y `<`), garantizando así la corrección lógica del programa.

Además, se ha evitado el uso de condiciones numéricas puras (como `if (a)`), dado que en Lisp podrían interpretarse como verdaderas incluso si no tienen el significado booleano deseado.

Validación

Se han probado múltiples variantes de estructuras `if`, incluyendo:

- Condicionales simples sin `else`
- Condicionales con `else` y múltiples sentencias por rama
- Anidamiento de `if` dentro de `if` y dentro de bucles

Todas las traducciones han generado código Lisp válido, libre de conflictos sintácticos y correctamente ejecutado en `clisp`. Se ha verificado además que el uso de `progn` evita errores cuando hay varias sentencias en las ramas.

3.8 Estructura de control **for**

El bucle **for** de C permite ejecutar un bloque repetidamente controlando tres fases: inicialización, condición y actualización. Su traducción a Lisp no es directa, por lo que se ha descompuesto semánticamente en una estructura equivalente basada en **loop** y **progn**.

Objetivo funcional

En Lisp no existe una construcción nativa idéntica al **for** de C. Por ello, el enfoque ha sido hacer algo similar a la estructura **while**, nos hemos basado en **descomponer el for en tres componentes**:

- **Inicialización**: ejecutada antes del bucle
- **Condición**: evaluada en cada iteración
- **Incremento**: ejecutado al final de cada iteración

Traducción estructurada

Un bucle como:

```
C/C++  
for (i = 0; i < 10; i = i + 1) {  
    printf("%d", i);  
}
```

se transforma en:

```
Unset  
(setq i 0)  
(loop while (< i 10) do  
  (progn  
    (princ i)  
    (setq i (+ i 1))  
  )  
)
```

Esta traducción separa claramente la inicialización y convierte el cuerpo del bucle en un bloque **progn**, que contiene tanto las sentencias del usuario como el incremento.

Reconocimiento gramatical

La producción gramatical reconoce exactamente los tres componentes del encabezado del **for**, separados por ; y encerrados entre paréntesis:

```
Unset  
"for" "(" asignacion ";" expresion ";" asignacion ")" bloque
```

Cada uno de estos componentes se almacena temporalmente. En la acción semántica asociada:

- La **inicialización** se imprime antes del bucle
- La **condición** se inserta en la instrucción `loop while`
- El **incremento** se añade al final del `do`, dentro de un `progn`

Esto permite reutilizar construcciones ya implementadas en la gramática (`asignación`, `expresión`, `bloque`), evitando duplicidad de reglas.

Consideraciones y restricciones

- No se admite declarar una nueva variable dentro del `for`, como `for (int i = 0; ...)`.
- El incremento debe ser una **asignación explícita** (`i = i + 1`), no se permiten operadores como `i++` o `i += 1`.
- El cuerpo del bucle siempre debe ir entre llaves `{}`.

Validación

Se han probado `for` con:

- Diferentes condiciones (`<=`, `!=`)
- Variación en la dirección del incremento (`i = i - 1`)
- anidarles estructuras complejas dentro de el bucle

Todas las versiones han producido código correcto en Lisp y han sido validadas en `clisp` sin errores.

3.9.1 Asignaciones en funciones y en `main`

Las asignaciones constituyen una de las operaciones fundamentales en los lenguajes imperativos como C. En nuestro traductor, las sentencias de asignación se reconocen dentro de funciones (`main` o auxiliares) y se transforman directamente en instrucciones `setf` de Lisp.

Comportamiento esperado

Una instrucción como:

```
C/C++  
a = 5 + b;
```

se traduce a:

```
Unset  
(setf a (+ 5 b))
```

Esto permite mantener el modelo imperativo de modificación de estado, equivalente al de C, pero adaptado a la semántica funcional de Lisp.

Reconocimiento gramatical

La producción para las asignaciones reutiliza la estructura:

```
Unset  
ID '=' expression ';' ;
```

Donde **ID** es el nombre de la variable y **expression** representa cualquier combinación válida de operadores aritméticos, lógicos o comparativos, ya definidos en la jerarquía de expresiones descrita en el punto 3.5.

Se restringe esta producción a los bloques de funciones (**main**, funciones auxiliares), no permitiéndose a nivel global (fuera de funciones).

Traducción semántica

Cada asignación genera una única línea de código Lisp con la estructura:

```
Unset  
(setf <variable> <expresión_traducida>)
```

Si la expresión es simple (**a = 3;**), se traduce de forma directa. Si es compuesta (**a = b + 4 * x;**), se aplica el sistema de precedencia ya gestionado por la gramática, obteniendo la forma prefija adecuada.

Consideraciones técnicas

- Solo se permiten asignaciones a variables ya declaradas.
- La asignación debe estar finalizada por **;**, y la variable debe estar en ámbito.
- Se admite el uso de asignaciones dentro de estructuras de control (**if**, **while**, **for**), incluso anidadas.

Ejemplo de asignaciones anidadas

```
C/C++  
if (x < 10) {  
    x = x + 1;  
}
```

Traducción resultante:

```
Unset  
(if (< x 10)
```

```
(setf x (+ x 1))  
)
```

Si se combinan con otras sentencias, se encapsulan automáticamente en un `progn`.

Validación

Se han probado asignaciones simples y compuestas, dentro de bloques `main`, funciones auxiliares y estructuras de control. Las salidas han sido correctas, con el uso adecuado de `setf` y sin conflictos de ámbito. También se han validado errores cuando se intenta usar una variable no declarada previamente.

3.9.2 Declaraciones en funciones y en `main`

En C, las variables locales se declaran dentro del cuerpo de las funciones, y su ámbito se restringe únicamente a dicho bloque. En nuestro traductor, estas variables también se representan mediante `setq`, pero con una modificación clave: concatenamos su identificador con el nombre de la función en la que se encuentran. Esta convención evita colisiones con variables globales y simula el comportamiento de ámbitos locales en un lenguaje como Lisp, donde `setq` define por defecto variables globales.

Reconocimiento gramatical

La gramática reconoce las declaraciones locales de la forma `int id = valor;` o `int id;` dentro de funciones. Estas se distinguen de las variables globales mediante el contexto: se activa una variable de control (`variables_locales`) que permite determinar si se está dentro del cuerpo de una función, concretamente dentro de `main` o una función auxiliar.

Para evitar conflictos con las declaraciones globales, se ha separado el reconocimiento de variables locales mediante reglas específicas como `vars_globales` para las variables globales y `lista_declaracion` para el tratamiento de las variables locales, garantizando así una estructuración jerárquica clara. Para poder haber introducido el nombre de la función se ha creado `funcion_actual` que guarda el nombre de la función cada vez que se entra a una de ellas.

Traducción semántica

Las variables locales se traducen a sentencias `(setq nombre_funcion valor)`. Por ejemplo:

C/C++

```
main() {  
    int a = 4;  
    a = a + 1;  
    printf("%d", a+1);  
}
```

Unset

```
(defun main ()  
  (setq main_a 4)  
  (setf main_a (+ main_a 1))  
  (princ (+ main_a 1))  
)
```

A partir de este punto, toda referencia a `a` dentro de `main` se transforma en `main_a`, tanto en asignaciones como en expresiones o llamadas a funciones.

Uso de `setf` en lugar de `setq`

Siguiendo la recomendación del enunciado, la instrucción `setf` se ha reservado para las asignaciones (modificación de valor) mientras que `setq` se usa exclusivamente para las declaraciones iniciales. Esta distinción mejora la legibilidad del código traducido y refleja de forma más clara el modelo imperativo de C.

Consideraciones técnicas

- El uso del guión bajo evita ambigüedades con el operador de resta `-`, tal como se especifica en el enunciado.
- Para implementar esta funcionalidad, se mantiene una tabla de símbolos local en cada función que registra los identificadores declarados como variables locales. Esta tabla permite saber cuándo debe aplicarse la concatenación.
- Si una variable usada en una función no está en la tabla local, se asume que es global y se traduce sin prefijo.

Validación

Se han probado múltiples escenarios con variables locales:

- Declaraciones múltiples dentro de `main` y funciones auxiliares.
- Combinación de variables locales y globales con el mismo nombre.
- Asignaciones y usos dentro de estructuras de control (`if`, `while`, `for`).

En todos los casos, la traducción a Lisp ha sido válida y funcional y comprobada mediante `clisp`

3.10 Definición de funciones auxiliares

Además de la función `main`, el lenguaje C permite definir múltiples funciones auxiliares. En nuestro traductor, estas funciones se reconocen y transforman en definiciones `defun` en Lisp, respetando el orden de aparición y su encapsulamiento estructural.

Reconocimiento gramatical

La gramática reconoce funciones con la siguiente estructura:

```
C/C++
nombreFuncion(param1, param2, ...) {
    <sentencias>
}
```

Esta estructura se traduce a Lisp como:

```
Unset
(defun nombreFuncion (param1 param2 ...)
    <cuerpo traducido>
)
```

Para ello, se diseñó una producción específica para funciones que no es `main`, pero que comparte gran parte de la estructura con ella. Esta producción:

- Reconoce el tipo `int` seguido de un `ID` distinto de `main`.
- Admite una lista de parámetros separados por comas.

Parámetros y cuerpo

La lista de parámetros se convierte en la lista de argumentos de `defun`, manteniendo su orden. Dentro del cuerpo se permiten las mismas sentencias que en `main`: asignaciones, estructuras de control (`if`, `while`, `for`), impresiones (`printf`, `puts`) y llamadas a otras funciones, incluyendo recursión.

Traducción semántica

C/C++

```
int suma(int a, int b) {  
    return a + b;  
}
```

se traduce a:

Unset

```
(defun suma (a b)  
  return-from suma (+ a b)  
)
```

En todo caso, las funciones devuelven un return. La estructura de los mismos es siempre la misma:

Unset

```
(return-from <funcion> (retorno))
```

Consideraciones técnicas

- No se permite declarar variables locales dentro de la lista de parámetros.
- El orden de aparición se mantiene. No se permite que `main` aparezca antes que funciones auxiliares.
- Todas las funciones se definen antes de que se ejecute (`main`).

Validación

Se han probado funciones:

- Sin parámetros
- Con múltiples parámetros
- Que devuelven un valor mediante `return`
- Que llaman recursivamente a sí mismas (ej: fibonacci)

En todos los casos, la generación de código Lisp fue correcta y se pudo ejecutar en `clisp` sin errores, cumpliendo los requisitos de la práctica.

3.11 Llamadas a funciones

En C, una función definida previamente puede ser invocada desde `main`, desde otras funciones, o incluso de manera recursiva. En nuestro traductor, las llamadas a funciones se reconocen y se transforman directamente a su forma prefija en Lisp, respetando el número y el orden de los parámetros.

Reconocimiento gramatical

La gramática implementada reconoce llamadas de la forma:

```
Unset
nombreFuncion(expr1, expr2, ...);
```

La producción utilizada permite capturar:

- Un identificador (`ID`) que corresponde a una función definida previamente.
- Una lista de expresiones separadas por comas, las cuales se traducen individualmente.
- Un punto y coma final obligatorio.

Traducción semántica

Cada llamada se traduce como una expresión prefija de tipo:

```
Unset
(nombreFuncion expr1 expr2 ...)
```

Esta traducción no necesita ningún operador adicional, ya que Lisp evalúa directamente las llamadas a funciones como cualquier otra expresión.

Consideraciones técnicas

- Las expresiones utilizadas como parámetros pueden ser simples (`a`) o compuestas `((+ a b))`.
- Las llamadas pueden formar parte de otras expresiones (por ejemplo, en una asignación o dentro de una condición), lo que permite anidamiento completo.

Ejemplo integrado en una asignación:

```
C/C++
res = suma(a, b);
```

Se traduce como:

```
Unset
(setf res (suma a b))
```

Uso dentro de estructuras

Las llamadas a funciones pueden incluirse dentro de cualquier bloque de código: `main`, funciones auxiliares, condicionales, bucles, o incluso dentro de otras llamadas. Esta flexibilidad se ha incorporado en la gramática permitiendo su reconocimiento como una sentencia válida.

Validación

Se han probado llamadas simples, múltiples, encadenadas, recursivas y combinadas con otras operaciones. En todos los casos, el código Lisp generado fue ejecutable correctamente en `clisp`, devolviendo los valores esperados. También se ha comprobado que se emiten errores si la función invocada no ha sido definida previamente o si se pasan argumentos incorrectos.

4. Memoria del Traductor Backend (`back.y`)

Este apartado debe explicar **cómo se traduce el código intermedio en Lisp** generado por el frontend (`trad.y`) hacia **código en notación Postfix (Forth-like)**, utilizando Bison.

4.1 Traducción de variables globales.

En Lisp, las variables globales generadas por el frontend se representan mediante expresiones (`setq identificador valor`). En el backend, estas se traducen directamente a notación Postfix de Forth, permitiendo inicializar dichas variables en memoria antes de su uso.

Reconocimiento gramatical

La gramática del backend reconoce esta estructura mediante:

```
Unset
sentencia:  SETQ IDENTIF expresion_parenthesis
```

Esta producción capta `setq`, `identificador` y `expresion_parenthesis`, que deriva en expresión y expresión con paréntesis por si se concatena con otras expresiones.

Traducción semántica

La acción semántica traduce el valor y el identificador a orden postfix, seguido del operador `!` de asignación de Forth. Por ejemplo, la entrada:

Unset

```
(setq x 7) - > variable x \n 7 x !
```

Esto guarda el valor 7 en la dirección asociada a la variable `x` en la pila de datos de Forth.

Consideraciones técnicas

Solo se permite asignación directa con constantes, ya que las expresiones más complejas se traducen en otras reglas posteriores (como `setf`). Además, esta instrucción debe aparecer antes de la ejecución de cualquier función, lo cual es coherente con su uso como declaración de variables globales.

4.2 Traducción de funciones (`main`)

En el código intermedio Lisp generado por el frontend, las funciones se representan con la sintaxis `(defun main () cuerpo)`.

Reconocimiento gramatical

Unset

```
sentencia : DEFUN MAIN '(')' lista_sentencias
```

Donde `DEFUN` identifica el inicio de una función, `MAIN` es el nombre de la función que vamos a reconocer, `lista_sentencias` permite acceder a sentencia de nuevo, pero también permite recursividad de sentencias con paréntesis para cada una de ellas. Una vez en `sentencia`, permite llamar a cualquiera de las producciones de nuevo dentro de la función de `main`.

Traducción semántica

La estructura `(defun main () cuerpo)` se traduce en Forth como:

Unset

```
: main( )  
  <cuerpo traducido>  
;
```

- Se inicia con `:` seguido del nombre de la función, en este caso solo se contempla `main`.

Validación

Se han realizado pruebas con funciones simples y recursivas, dentro de la misma, generadas por el frontend. La ejecución en **gforth** ha sido correcta.

Entrada Lisp final:

```
Unset
(defun main ()
  (princ (b))
)
se traduce a:

: main
b @ .
;
```

Además, como bien indica el enunciado de la práctica, la ejecución del programa debe activarse mediante una llamada explícita a **(main)** en el código Lisp, que proviene directamente de la directiva especial **//@ (main)** escrita en C. Esta sentencia **(main)** es procesada por el backend como una llamada normal a función, y se traduce simplemente como:

```
Unset

main
```

Gracias a esta llamada explícita al final del programa, la ejecución de la función **main** se lanza correctamente al cargar el código en **gforth**.

4.3 Traducción de expresiones aritméticas, comparaciones y operadores lógicos

El backend reconoce expresiones aritméticas, comparaciones y operadores lógicos escritas en notación prefija Lisp y las convierte a notación postfija, tal como exige el lenguaje Forth. Este proceso se realiza de forma directa y recursiva, evaluando los operandos en profundidad antes de aplicar el operador correspondiente.

Reconocimiento gramatical

Las expresiones se reconocen en la producción expresión, mediante reglas específicas para cada operador, por ejemplo:

Unset

```
expresion:  
    '+' expresion_parentesis expresion_parentesis {  
        sprintf (temp, "%s %s +", $2.code, $3.code }  
    }
```

Cada operador requiere exactamente dos operandos, salvo operadores unarios como **not**, que admiten uno solo. Los operandos pueden ser números, variables o subexpresiones complejas. Las reglas utilizan una producción auxiliar **expresion_parentesis** que permite admitir expresiones con o sin paréntesis externos, proporcionando mayor flexibilidad al parser.

Traducción semántica

La traducción se realiza concatenando las traducciones de los operandos en orden, seguidas del operador Forth correspondiente. Esto refleja la semántica de evaluación de una máquina de pila, en la que los operandos se empujan primero y luego se aplica la operación.

Ejemplos representativos

Unset

```
(+ 2 4) -> 2 4 + | (* (+ 1 2) 3) -> 1 2 + 3 * |  
(= x 5) -> x @ 5 = | (not (= x 3)) -> x @ 3 = 0=  
(and (< x 10) (> x 0)) -> x @ 10 < x @ 0 > and
```

La evaluación se realiza de forma recursiva desde las hojas del árbol hacia la raíz, siguiendo el orden natural de una máquina de pila.

Consideraciones técnicas

- Los operandos se traducen usando la producción **operando**, que distingue entre variables (**IDENTIF**) y números (**NUMBER**).
- Las variables se traducen como accesos a memoria mediante el operador **@**, por ejemplo: $x \rightarrow x @$.
- El operador **DISTINTO** (**!=**) se implementa como **= 0=**, y el operador **not** también se traduce mediante **0=**, dado que en Forth se invierte el valor lógico de esta forma.
- Todos los operadores lógicos (**and**, **or**, **not**) y de comparación (**=**, **<**, **>**, **<=**, **>=**, **/=**) están cubiertos explícitamente por la gramática.

Validación

Se han probado múltiples combinaciones de expresiones aritméticas, comparativas y lógicas, incluyendo casos anidados, distintas prioridades operativas y mezclas de constantes y variables. Todas las traducciones han sido validadas correctamente en **gforth**, confirmando que la pila se comporta como se espera y que el orden de evaluación es correcto.

4.4 Traducción de asignaciones locales (**setf**)

En el código Lisp generado por el frontend, las asignaciones a variables (tanto locales como globales) se representan mediante la forma (**setf id expr**). En el backend, esta instrucción se traduce directamente a notación postfija de Forth, respetando el modelo de ejecución en pila.

Reconocimiento gramatical

La producción correspondiente en la gramática del backend es:

Unset

```
sentencia: SETF IDENTIF expresion_parenthesis {  
    sprintf(temp, "%s %s !", $3.code, $2.code);  
    $$ .code = gen_code(temp);  
}
```

Esta regla capta la palabra clave **setf**, el identificador de la variable, y una expresión entre paréntesis o sin ellos. La expresión puede ser una constante, una variable u otra expresión compleja.

Traducción semántica

La acción semántica empuja primero el valor evaluado de la expresión, seguido del identificador de la variable, y aplica el operador **!** de Forth, que guarda el valor en la dirección de memoria de dicha variable.

Ejemplo:

Unset

```
(setf main_a (+ 2 3)) -> 2 3 + main_a !
```

Este código evalúa la suma `2 + 3` y almacena el resultado en la variable `main_a`, que previamente debe haber sido declarada mediante `setq`.

Gestión de variables locales

Aunque en Forth no existe una distinción explícita entre variables locales y globales, en nuestro backend se conserva la semántica del frontend, donde las variables locales en funciones (como `main`) llevan prefijos como `main_`. Esto evita colisiones de nombres y mantiene una simulación de ámbito similar al de C.

La gramática no comprueba si la variable ha sido declarada con `setq`, pero se asume que el frontend genera siempre un `setq` antes de cualquier `setf`, incluso para variables locales.

Consideraciones técnicas

- Solo se permiten asignaciones a variables ya declaradas.
- No se gestionan ámbitos dinámicos ni estructuras locales complejas como `let`.
- No se admite asignación a subcomponentes (por ejemplo, vectores).
- Las expresiones asignadas pueden ser simples o compuestas, siempre que sean reconocidas por la producción `expresion`.

Validación

Se han realizado pruebas con asignaciones simples y compuestas dentro del cuerpo de `main`, incluyendo operaciones aritméticas, llamadas a funciones y estructuras de control. Todas las asignaciones fueron traducidas correctamente a Forth y evaluadas con éxito en `gforth`, almacenando los resultados esperados en memoria.

4.5 Traducción de `if` y `if` con `else`

Las estructuras condicionales en Lisp se representan mediante expresiones del tipo `(if cond cuerpo1)` o `(if cond cuerpo1 cuerpo2)`, equivalentes a las estructuras `if` y `if-else` del lenguaje C. El backend traduce estas construcciones directamente a la notación de control de flujo de Forth utilizando los operadores `IF`, `ELSE`, `THEN`. Se ha tenido en cuenta el caso genérico por lo que siempre vienen con `progn` para diferenciar bien el `if` y el `else`

Reconocimiento gramatical

La producción encargada de reconocer estas estructuras en la gramática es:

Unset

```
if: IF expresion_parenthesis sentencia_if sentencia_else {  
    if (strlen($4.code) <= 0) {
```

```
        sprintf(temp, "%s\nIF\n%s\nTHEN", $2.code, $3.code);  
    } else {  
        sprintf(temp, "%s\nIF\n%s\nELSE\n%s\nTHEN",  
$2.code, $3.code, $4.code);  
    }  
    $$code = gen_code(temp);  
}
```

- `expresion_parentesis` evalúa la condición.
- `sentencia_if` y `sentencia_else` contienen los bloques `then` y `else`, respectivamente.
- Si no hay bloque `else`, se omite automáticamente.

Además, la gramática permite que cada bloque sea una única sentencia o un `progn` con múltiples instrucciones:

Unset

```
sentencia_if: '(' PROGN lista_sentencias ')' | '(' sentencia ')'  
sentencia_else: '(' PROGN lista_sentencias ')' | '(' sentencia  
)' | /* lambda */
```

Esto ofrece flexibilidad para estructuras simples y compuestas.

Traducción semántica

La traducción convierte la expresión condicional en una estructura `IF ... ELSE ... THEN` en Forth. Si no hay bloque `else`, se omite `ELSE`.

Ejemplo sin else:

Unset

```
(if (< x 10)  
    PROGN ((setf x (+ x 1))))
```

se traduce a:

```
x @ 10 <  
IF  
x @ 1 + x !  
THEN
```

Ejemplo con else:

```
Unset

(if (= x 0)
  PROGN ((print "es cero"))
  PROGN ((print "no es cero"))
)
se traduce a:

x @ 0 =
IF
." es cero"
ELSE
." no es cero"
THEN
```

Consideraciones técnicas

- La condición debe ser una expresión lógica válida, traducida previamente por las reglas de **expresion**.
- Las instrucciones de los bloques se traducen como una secuencia de sentencias, sin necesidad de anidar IF dentro de IF.
- En Forth, cualquier valor distinto de 0 es verdadero (**-1** típicamente), y **0** es falso.

Validación

Se han validado condiciones simples, combinadas y anidadas, incluyendo estructuras como:

- **if** sin **else**
- **if** con **else** y múltiples sentencias en ambos bloques
- Condiciones que implican comparaciones y operadores lógicos

Todas las traducciones generadas por el backend han producido código válido y funcional en **gforth**, respetando el flujo condicional esperado y sin errores de alineación en **IF**, **ELSE** y **THEN**.

4.6 Traducción de bucles **while**

En el código intermedio generado por el frontend, los bucles **while** en C se traducen a expresiones Lisp de la forma **(loop while cond do cuerpo)**. En el backend, estas estructuras se transforman directamente en bucles de tipo **BEGIN ... WHILE ... REPEAT** en Forth, adaptándose al funcionamiento típico de una máquina de pila.

Reconocimiento gramatical

La producción encargada de reconocer este patrón en la gramática es:

```
Unset
sentencia: LOOP WHILE expresion_parentesis DO lista_sentencias
{
    sprintf(temp, "BEGIN\n%s\nWHILE\n%s\nREPEAT", $3.code,
$5.code);
    $$code = gen_code(temp);
}
```

- **Expresion_parentesis** corresponde a la condición del bucle.
- **lista_sentencias** recoge las instrucciones que componen el cuerpo del bucle.
- El uso de **BEGIN**, **WHILE**, **REPEAT** en Forth garantiza un control de flujo adecuado.

Traducción semántica

La condición del bucle se evalúa al inicio de cada iteración. Si se cumple (es decir, es distinta de cero), se ejecuta el cuerpo del bucle; de lo contrario, se abandona.

Ejemplo Lisp:

```
Unset

(loop while (< x 10) do
    (setf x (+ x 1)))
se traduce a:
BEGIN
x @ 10 <
WHILE
x @ 1 + x !
REPEAT
```

Esto refleja la lógica habitual de un bucle **while**, donde la condición se comprueba antes de entrar al cuerpo del bucle y tras cada iteración.

Consideraciones técnicas

- La condición debe ser una expresión lógica correctamente formada.
- El cuerpo del bucle puede contener una o varias sentencias, encapsuladas por la producción `lista_sentencias`.
- No se permite `loop while` con condiciones no booleanas o sin un bloque `do` definido.
- Se ha cuidado el orden de evaluación en la generación del código, asegurando que la condición se evalúa antes del `WHILE`, tal como requiere Forth.
- Como la traducción de c a lisp de un for se pone como un while con una declaración y un incremento en el back se traduce como una expresión loop

Validación

Se han probado bucles simples y anidados con distintas condiciones, incrementos y salidas. También se han verificado casos límite como:

- Condición inicialmente falsa (no ejecuta el cuerpo)
- Incrementos dentro del cuerpo del bucle
- Combinación con instrucciones `if`
- bucles anidados

Todas las pruebas han producido código funcional en `gforth`, validando tanto la estructura como el comportamiento esperado del bucle.

4.7 Traducción de `print`, `princ`

En el backend, las instrucciones de impresión generadas por el frontend Lisp se traducen directamente a su equivalente en Forth, utilizando el operador `.` para mostrar por pantalla. Se distingue entre las funciones Lisp `(print ...)` y `(princ ...)`

Traducción y diferencias

Unset
`(print "Hola") → "Hola"`

Simula el uso de `"puts"` en C. Imprime una sola cadena de caracteres.

Unset

```
(princ x) → x .
```

Simula el uso de “`printf`” en C, pudiendo imprimir varios elementos donde entran otro tipo de sentencias, aparte de los string.

Implementación

La implementación correspondiente para soportar las reglas del princ son:

Unset

```
sentencia:      | PRINC expresion_parentesis  
                | PRINC STRING
```

Y la referente al print está compuesta por la siguiente sentencia:

Unset

```
sentencia: | PRINT STRING
```

Validación

Ambos casos han sido probados en gforth, verificando que las cadenas se imprimen correctamente y con el formato esperado, cumpliendo con el comportamiento de impresión definido en Lisp.

5. Validación y pruebas

Para comprobar la correcta funcionalidad del traductor completo, se ha llevado a cabo un proceso sistemático de pruebas sobre el conjunto de código generado por el frontend (`trad.y`) y traducido por el backend (`back.y`).

Como base de referencia, se han utilizado los casos de prueba proporcionados por el profesorado en la carpeta `test.zip`, disponible en el Aula Global. Estos ejemplos cubren una amplia variedad de construcciones: declaraciones de variables, funciones, estructuras de control, expresiones aritméticas y operaciones de impresión.

A partir de esas pruebas iniciales, se han creado versiones propias con modificaciones y ampliaciones, adaptadas a nuestro estilo de desarrollo. Estas nuevas pruebas mantienen la misma intención funcional que las originales, pero incluyen detalles personalizados como el uso de nombres distintos, funciones recursivas o combinaciones más complejas de estructuras.

Todos los casos han sido ejecutados y validados correctamente en `clisp` (tras el paso del frontend) y en `gforth` (una vez procesados por el backend), sin errores de compilación ni ejecución. Se ha comprobado que las salidas producidas son correctas, y que el comportamiento general del traductor es robusto y coherente con los requisitos definidos en el enunciado.

6. Conclusión final

La realización de este proyecto ha supuesto la implementación completa de un traductor dividido en dos fases: un frontend que convierte código en lenguaje C a una representación intermedia en Lisp, y un backend que traduce dicho código Lisp a notación Postfix ejecutable en Forth. Este enfoque modular nos ha permitido trabajar de forma estructurada, siguiendo una lógica de compilación por etapas que reproduce el funcionamiento interno de compiladores reales.

Durante el desarrollo, se ha prestado especial atención al cumplimiento estricto de las especificaciones indicadas en la práctica. La gramática del frontend se ha diseñado con una jerarquía clara, evitando conflictos y asegurando que cada construcción sintáctica se traduzca de forma limpia y semánticamente coherente. En paralelo, el backend ha sido implementado como un sistema directo que procesa las instrucciones de forma inmediata y genera una salida Postfix válida y ejecutable.

A lo largo del proceso se han empleado herramientas como Bison y Flex, así como entornos de prueba (`clisp` y `gforth`) que han permitido validar el comportamiento final del traductor con éxito. Las pruebas diseñadas, tanto las ofrecidas por el profesorado como las ampliadas por nosotros, han demostrado la robustez y corrección del sistema implementado.

Consideramos que el resultado final refleja fielmente los objetivos del proyecto y constituye una implementación completa, funcional y alineada con los principales objetivos del proyecto.